EAST WEST UNIVERSITY

Course Details

Course Title: Data Mining

Couse Code: CSE 477

Section: 02

Submitted To

Amit Mandal
Senior Lecturer
Department of Computer Science and Engineering

Submitted By

Farhatun Nahar Priya
Student ID: 2023-1-60-202

Date of Submission: 25th February 2026.

Table of Contents

1. Introduction

In today's digital age, YouTube serves as a major platform where users express their thoughts, opinions, and reactions through comments. These comments form a rich source of textual data that can reveal emerging trends, audience sentiments, and relationships between topics over time. Analyzing such data incrementally chunked by chunk allows us to understand not just what is popular, but how patterns evolve as new content and discussions emerge.

This lab focuses on incremental pattern mining, a technique that identifies frequent words (unigrams) and co-occurring word pairs within sequential chunks of YouTube comments. By tracking these patterns over multiple time segments, we can observe which topics gain or lose prominence, detect recurring associations, and explore correlations between words that reflect the context of the discussions.

Additionally, correlation analysis is employed to understand the relationships between selected word patterns, providing insights into how certain topics influence or relate to one another. Overall, this study demonstrates the power of combining frequency tracking and correlation analysis to uncover meaningful trends in textual data, making it a practical approach for social media analytics, content strategy, and audience behavior studies.

2. Objective

The objective of this lab is to analyze YouTube comments to uncover meaningful patterns and relationships over time. Specifically, it aims to identify frequent words and word pairs, track how these patterns evolve across sequential data chunks, and examine correlations between selected patterns. This approach provides insights into the dynamics of audience interactions and emerging trends in online discussions, demonstrating how incremental pattern mining and correlation analysis can reveal temporal and contextual patterns in textual data.

3. Tools and Libraries Used

3.1 Development Environment

This lab was implemented using the Kaggle Notebook Environment, which provides an integrated platform for writing, executing and visualizing Python based data analysis workflows. The environment allows direct dataset access, code execution and inline visualization of plots and tables.

The analysis was conducted using Python (Version 3.10), as it provides extensive library support for data manipulation, frequency-based analysis and visualization.

3.2 Python Libraries and Their Roles

Several standard Python libraries were used to complete different stages of the analysis:

1. pandas

The pandas library was used for structured data handling and analysis. Its primary roles in this lab included:

- Loading the dataset using read_csv()
- Managing and manipulating DataFrames
- Constructing summary tables for unigram and bigram comparisons
- Creating a frequency series DataFrame across chunks
- Computing the correlation matrix between selected patterns

This library formed the backbone of the data processing workflow.

2. numpy

The numpy library was used to simulate incremental data arrival by dividing the dataset into five equal sized chunks using numpy.array_split(). This allowed the analysis to examine how frequent patterns evolved across different segments of the dataset.

3. collections(Counter)

The Counter class from the collections module was used for frequency counting. It was applied to:

- Count individual token frequencies (unigrams)
- Count co-occurring word pairs (bigrams)

This enabled efficient identification of the most frequent patterns in each chunk.

4.itertools(combinations)

The combinations function from the itertools module was used to generate unordered word pairs from each comment's token list. This ensured that co-occurring word pairs were extracted without duplication and treated consistently across the dataset.

5.mathplotlib.pyplot

The matplotlib library was used to generate visual representations of the results, including:

- Bar charts for top unigrams in each chunk
- Bar charts for top co-occurring word pairs
- Line plots to illustrate frequency changes of selected patterns across chunks

These visualizations helped identify trends and pattern evolution.

6. seaborn

The seaborn library was used to create a correlation heatmap between selected patterns. This provided a clearer understanding of the relationships and co-variation between frequently occurring words and word pairs.

7. ast

The ast module was used to convert string-formatted token lists into actual Python lists using ast.literal_eval(). This ensured proper data type handling before performing frequency analysis.

3.3 Overall Technical Approach

The lab relied exclusively on core Python data science libraries without using advanced pattern mining frameworks. The implementation focused on:

- Structured data manipulation
- Frequency-based pattern extraction
- Incremental chunk analysis
- Visualization-driven interpretation
- Correlation-based relationship analysis

This approach ensured clarity, reproducibility and alignment with the objectives of incremental frequent pattern mining.

4. Dataset Description

4.1 Dataset Origin and Context

The dataset used in this lab, **cleaned_comments.csv**, was generated from Lab 2 after applying systematic preprocessing to raw YouTube comments. The preprocessing steps included tokenization, converting text to lowercase, and removing noise such as punctuation, special characters, and common stopwords. This ensures that the textual data is clean, structured, and ready for analysis.

Starting with this cleaned dataset allows the focus to remain on pattern discovery rather than text cleaning, improving analytical consistency and computational efficiency.

```
df = pd.read_csv('/kaggle/input/datasets/priyaa652/cleaned-comments/cleaned_comments.csv')
df.head()
```

	username	timestamp_text	comment_text	cleaned_tokens
0	@milojadez	14 hours ago	There are people out there that actually are a...	['people', 'actually', 'able', 'read', 'mind',...
1	@JasonWestaway	22 hours ago	Wow, that was insane! 1	['wow', 'insane']
2	@THEHONESTREPLAY	1 day ago	That's all are fake, Edited, i challenged,	['thats', 'fake', 'edited', 'challenged']
3	@serenaLei06	1 day ago	Wait, how did he do that?	['wait']
4	@ainomugishanancy206	1 day ago	If i know how people think,i know what they th...	['know', 'people', 'thinki', 'know', 'think']

Figure 1: Loading the cleaned_comments.csv dataset and previewing the first five rows using df.head().

4.2 Data Structure

Each row in the dataset corresponds to a single YouTube comment. The main column used for analysis is:

- **cleaned_tokens** : a list of processed words extracted from each comment.

The tokens were standardized through:

- Lowercasing
- Removal of punctuation and special characters
- Stopword filtering
- Tokenization

This structured representation transforms unstructured text into a transaction-like format suitable for frequency mining.

4.3 Treating Comments as Transactions

For frequent pattern mining, each comment is treated as an individual transaction, while the words in cleaned_tokens are considered items.

This approach aligns with the classical market-basket model in data mining:

- Comment: Transaction
- Words in the comment: Items

Such a representation enables identification of:

- Frequently occurring individual words (unigrams)
- Words that co-occur frequently (bigrams)

4.4 Simulating Incremental Data

To examine how patterns may evolve over time, the dataset was divided into five roughly equal segments using numpy.array_split().

Each segment represents a simulated stage of data arrival. Pattern mining on each segment can reveal:

- Stability of frequent patterns
- Emerging or declining trends
- Changes in word co-occurrence behavior

This method mirrors incremental data growth commonly observed in social media platforms.

Figure 2: Splitting DataFrame into 5 chunks

5. Methodology / Procedure

This section explains the step-by-step process followed in this experiment. It includes loading and segmenting the cleaned dataset, analyzing frequent words and word pairs in each chunk, tracking how these patterns change over time and performing correlation analysis to understand relationships between selected patterns. Finally, insights are drawn based on the observed trends.

5.1 Data Loading and Segmentation

Here, the cleaned YouTube comment dataset (cleaned_comments.csv) generated in Lab 2 was used as the primary data source. This file contains one row per comment and includes a cleaned_tokens column that stores the preprocessed tokens extracted during the previous lab.

The dataset was loaded using the pandas library:

```
df = pd.read_csv('/kaggle/input/datasets/priyaa652/cleaned-comments/cleaned_comments.csv')
df.head()
```

Figure 3: Loading the cleaned_comments.csv dataset

The cleaned_tokens column is essential for pattern mining because it represents each comment as a list of cleaned words. However, depending on how the file was saved, these token lists may sometimes be stored as string representations rather than actual Python lists. To ensure consistency and avoid processing errors, a type check was performed. If the entries were strings, they were converted into proper Python lists using ast.literal_eval():

```
if isinstance(df['cleaned_tokens'].iloc[0], str):
    df['cleaned_tokens'] = df['cleaned_tokens'].apply(ast.literal_eval)
df['cleaned_tokens'].iloc[0]
```

```
['people',
 'actually',
 'able',
 'read',
 'mind',
 'solve',
 'murder',
 'unexplainably',
 'guy',
 'found',
 'way',
 'gain']
```

Figure 4: Converting stringified tokens to Python lists using ast.literal_eval()

This step ensures that each comment is correctly treated as a transaction (a list of tokens), which is required for accurate frequency counting and pattern mining.

Data Segmentation

To simulate the incremental arrival of data as occurs in real-world comment streams the dataset was divided into five approximately equal segments using NumPy:

```
chunks = np.array_split(df, 5)
len(chunks)
```

```
5
```

Figure 5: Splitting DataFrame into 5 chunks

Each chunk represents roughly 20% of the total dataset. These chunks were treated as sequential batches of incoming data. Pattern mining was performed separately on each chunk to observe how word frequencies and associations evolve as more data becomes available.

To verify the segmentation, the total number of chunks and their respective sizes were printed. The output shows that the dataset was successfully divided into five chunks, each containing 94 records, ensuring a balanced distribution across all segments.

```
chunks = np.array_split(df, 5)

print("Number of chunks:", len(chunks))
print("Chunk sizes:", [len(chunk) for chunk in chunks])
```

```
Number of chunks: 5
Chunk sizes: [94, 94, 94, 94, 94]
```

Figure 6: Output showing the total number of chunks and their respective sizes after dataset segmentation.

5.2 Pattern Mining per Chunk

For each of the five chunks, pattern mining was performed independently to observe how frequent patterns evolve across simulated time.

The cleaned token list of each comment was treated as a transaction:

```
transactions = chunk['cleaned_tokens'].tolist()
```

Each transaction represents the set of cleaned words contained in a single comment. This transaction-based structure allows frequency analysis and co-occurrence mining within each chunk.

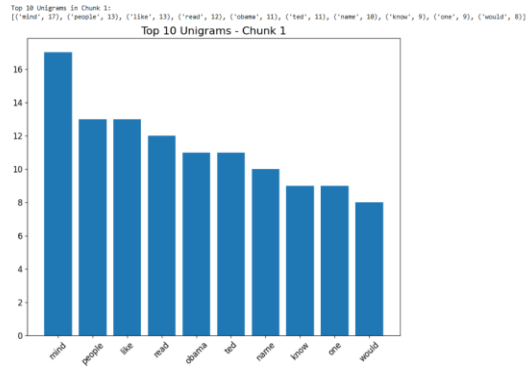
To identify the most frequent individual words (unigrams), the Counter class from the collections module was used:

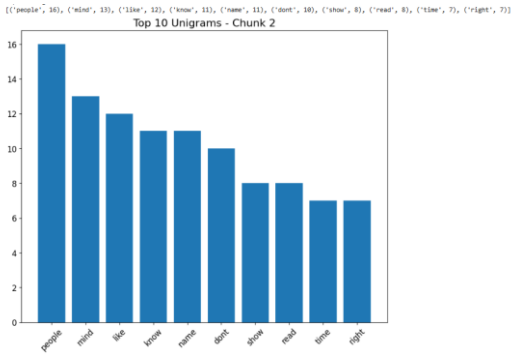
```
unigram_counts = Counter([token for row in transactions for token in row])
top_unigrams = unigram_counts.most_common(10)
```

This process counts all tokens appearing in the chunk and extracts the top 10 most frequent words. These words indicate the dominant vocabulary and main themes present in that segment of the dataset.A bar chart was generated to visualize the frequency distribution:

```
plt.bar([w for w, _ in top_unigrams], [c for _, c in top_unigrams])
```

This visualization makes it easier to identify which words are most prominent within each time segment.





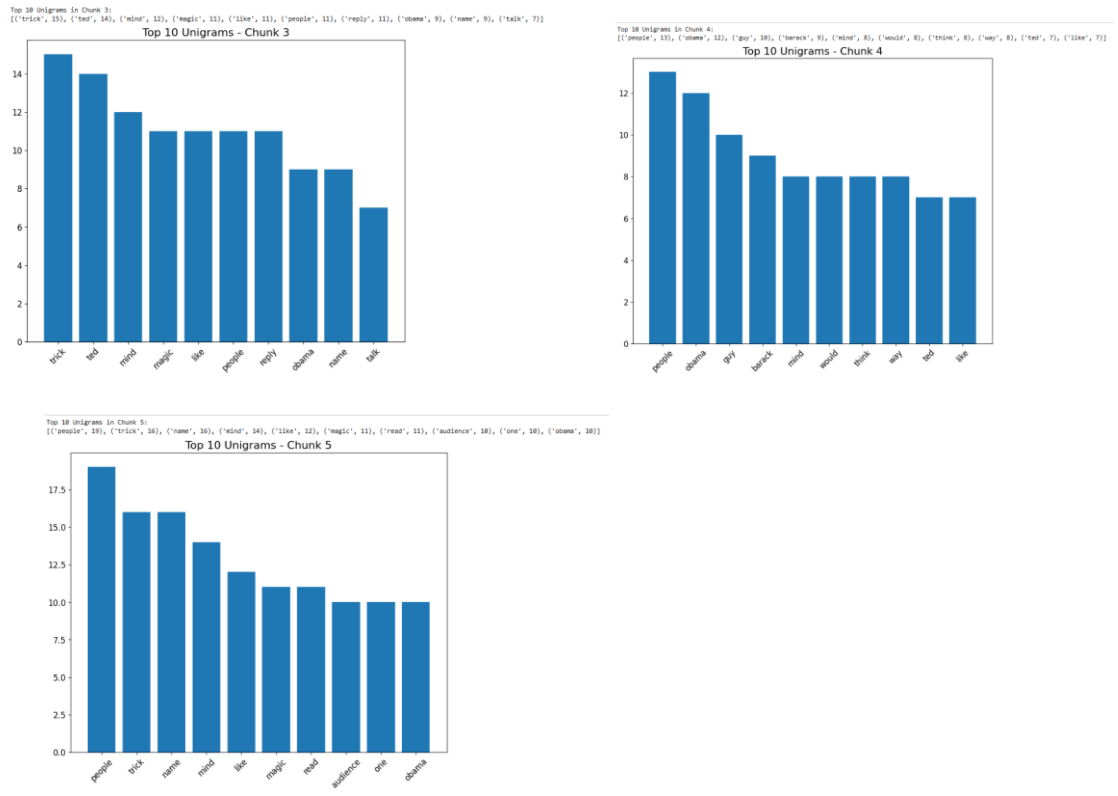


Figure 7: Bar charts showing top unigrams across all 5 chunks

In addition to individual word frequencies, co-occurring word pairs were identified to explore thematic associations. Unordered word pairs were generated using itertools.combinations():

```
bigram_counts = Counter()
for row in transactions:
    bigram_counts.update(combinations(sorted(set(row)), 2))
```

This approach counts word pairs that appear together in the same comment. The top 10 most frequent co-occurring pairs were then extracted:

```
top_bigrams = bigram_counts.most_common(10)
```

Bar charts were created to visualize the frequency of these word pairs.

These pairs represent unordered co-occurring itemsets rather than adjacent linguistic bigrams. In this method, two words are considered related if they appear within the same comment,

regardless of their order or position. This approach helps identify broader thematic relationships rather than exact phrase patterns.

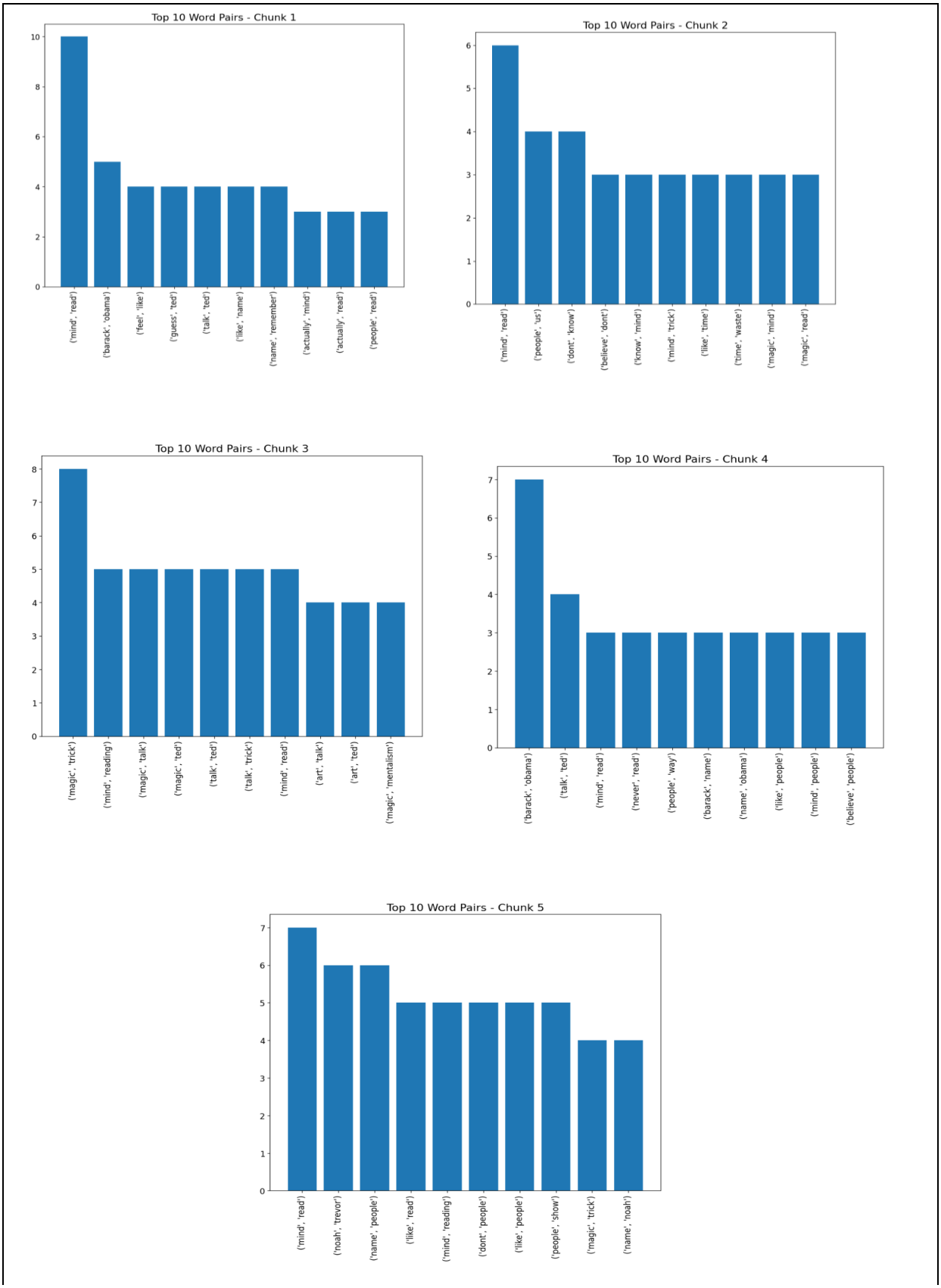


Figure 8: Co-occurrence patterns of top unigrams in all chunks

5.3 Tracking Pattern Evolution

To analyze how textual patterns evolve across the dataset, a set of representative unigrams and co occurring word pairs was selected for detailed tracking. These patterns were chosen based on their thematic relevance and recurrence in chunk level analysis. Tracking these patterns across chunks allows temporal-style observation of word usage and conceptual associations.

The selected patterns include three unigrams and two co-occurring word pairs:

```
patterns = [
    'mind',
    'people',
    'like',
    ('mind', 'read'),
    ('magic', 'trick')
]
```

These patterns were tracked across all five chunks to observe how their frequencies change over simulated time. Frequency data for each selected pattern was extracted from the chunk level counts and compiled into a pandas DataFrame:

```
freq_data = {}

for word in patterns:
    if isinstance(word, str):  # unigram
        freq_data[word] = [counts.get(word, 0) for counts in unigram_data[:5]]
    else:
        freq_data[f"{word[0]}, {word[1]}"] = [counts.get(word, 0) for counts in bigram_data[:5]]

freq_df = pd.DataFrame(freq_data, index=[f"Chunk {i+1}" for i in range(5)])

print("Frequency Series of Selected Patterns Across 5 Chunks:")
display(freq_df)
```

This produced the following summary table:

Frequency Series of Selected Patterns Across 5 Chunks:

	mind	people	like	mind, read	magic, trick
Chunk 1	17	13	13	10	0
Chunk 2	13	16	12	6	1
Chunk 3	12	11	11	5	8
Chunk 4	8	13	7	3	2
Chunk 5	14	19	12	7	4

Figure 9: Frequency trends of selected patterns across 5 chunks

- Each row represents a chunk and each column represents a selected pattern.
- Frequency values indicate the number of occurrences of that pattern in the corresponding chunk.
- Patterns absent in a chunk are recorded as 0, ensuring consistent comparison.

To extend this analysis beyond the selected patterns, the top 10 unigrams and top 10 co-occurring pairs from each chunk were aggregated to create comprehensive summary tables:

```
# Unigram summary
unigram_summary = pd.DataFrame(index=sorted(all_top_unigrams))

# Co-occurring pair summary
pair_summary = pd.DataFrame(index=sorted(all_top_pairs))
```

Unigram Comparison Across 5 Chunks:

	Chunk 1	Chunk 2	Chunk 3	Chunk 4	Chunk 5
audience	0	2	3	3	10
barack	5	3	4	9	6
dont	3	10	7	5	6
guy	7	6	5	10	8
know	9	11	6	5	4
like	13	12	11	7	12
magic	2	7	11	6	11
mind	17	13	12	8	14
name	10	11	9	5	16
obama	11	5	9	12	10
one	9	1	3	1	10
people	13	16	11	13	19
read	12	8	7	5	11
reply	0	4	11	5	4
right	0	7	2	1	1
show	3	8	6	4	9
talk	7	4	7	5	5
ted	11	5	14	7	6
think	3	2	6	8	4
time	2	7	7	3	5
trick	2	5	15	6	16
way	1	3	2	8	7
would	8	4	4	8	8

Figure 10: Frequency comparison of top unigrams across five chunks

Co-occurring Pair Comparison Across 5 Chunks:

	Chunk 1	Chunk 2	Chunk 3	Chunk 4	Chunk 5
(actually, mind)	3	0	1	1	0
(actually, read)	3	0	1	0	0
(art, talk)	0	0	4	0	0
(art, ted)	0	0	4	0	0
(barack, name)	1	0	0	3	1
(barack, obama)	5	3	4	7	4
(believe, dont)	1	3	2	2	2
(believe, people)	0	0	2	3	1
(dont, know)	0	4	2	2	1
(dont, people)	0	1	1	1	5
(feel, like)	4	1	2	0	2
(guess, ted)	4	0	0	0	0
(know, mind)	3	3	1	0	1
(like, name)	4	2	1	0	3
(like, people)	1	3	3	3	5
(like, read)	1	1	0	0	5
(like, time)	1	3	1	2	2
(magic, mentalism)	0	0	4	0	0
(magic, mind)	0	3	4	0	1
(magic, read)	0	3	0	0	1
(magic, talk)	0	1	5	0	1
(magic, ted)	0	1	5	0	2
(magic, trick)	0	1	8	2	4
(mind, people)	2	1	3	3	3
(mind, read)	10	6	5	3	7
(mind, reading)	0	2	5	2	5
(mind, trick)	0	3	4	0	2
(name, noah)	0	0	1	0	4
(name, obama)	2	0	0	3	1
(name, people)	3	1	1	0	6
(name, remember)	4	2	2	0	2
(never, read)	0	0	0	3	0
(noah, trevor)	3	2	2	2	6
(people, read)	3	2	3	2	4
(people, show)	0	2	0	1	5
(people, us)	0	4	0	1	1
(people, way)	1	0	0	3	3
(talk, ted)	4	2	5	4	4
(talk, trick)	0	2	5	1	0
(time, waste)	0	3	0	0	0

Figure 11: Frequency comparison of top co-occurring pair across five chunk

- The unigram summary table tracks the frequency of each unique word across all chunks.
- The co-occurring pair table tracks each frequent word pair across chunks.
- Absent words or pairs in a chunk are recorded as 0, preserving table integrity and enabling clear comparison.

These tables allow detailed observation of pattern evolution, including:

- Words that remain consistently frequent across multiple chunks.
- Patterns that emerge in later chunks, indicating new topics.
- Words or pairs that decline or disappear over time.

By organizing frequency data in this structured, comparative tabular format, the analysis provides a clear, temporal view of pattern evolution, supporting subsequent interpretation and correlation analysis.

5.4 Correlation Analysis

To examine the relationships between selected patterns over time, 5 meaningful patterns were chosen based on their frequency variation across chunks. These patterns include:

- mind
- people
- like
- mind, read
- magic, trick

These patterns were selected because:

1. Some are core topic-related words representing central discussion themes.
2. Some capture co-occurring thematic associations.
3. Some exhibit noticeable changes in frequency across chunks, making them suitable for correlation analysis.

Frequency Series Construction

For each selected pattern, its frequency across the five chunks was extracted from the previously computed chunk-level data and organized into a pandas DataFrame:

```
freq_df = pd.DataFrame(freq_data, index=[f"Chunk {i+1}" for i in range(5)])
```

This produces a frequency table where:

- Each row represents a chunk.
- Each column represents a pattern.
- Each value indicates the frequency of that pattern in the corresponding chunk.

	mind	people	like	mind, read	magic, trick
Chunk 1	17	13	13	10	0
Chunk 2	13	16	12	6	1
Chunk 3	12	11	11	5	8
Chunk 4	8	13	7	3	2
Chunk 5	14	19	12	7	4

Table 1: Frequency distribution of selected unigrams and bigrams across five chunks

Correlation Matrix

To quantify how patterns co-vary over time, a correlation matrix was computed using pandas:

```
corr = freq_df.corr()
print("Correlation Matrix:")
print(corr)
```

The correlation matrix provides numerical values between -1 and 1:

- +1.0: Strong positive correlation (patterns rise and fall together)
- -1.0: Strong negative correlation (one rises while the other falls)
- 0: No clear relationship

	mind	people	like	mind, read	magic, trick
mind	1.00	0.23	0.95	0.98	-0.27
people	0.23	1.00	0.31	0.20	-0.25
like	0.95	0.31	1.00	0.86	-0.10
mind, read	0.98	0.20	0.86	1.00	-0.40
magic, trick	-0.27	-0.25	-0.10	-0.40	1.00

Table 2: Correlation matrix of selected unigrams and bigrams across five chunks

Correlation Heatmap Visualization

A heatmap provides a visual summary of correlations:

```
plt.figure(figsize=(10,8))
sns.heatmap(freq_df.corr(), annot=True, fmt=".2f")
plt.title("Correlation Heatmap of Selected Patterns", fontsize=16)
plt.show()
```

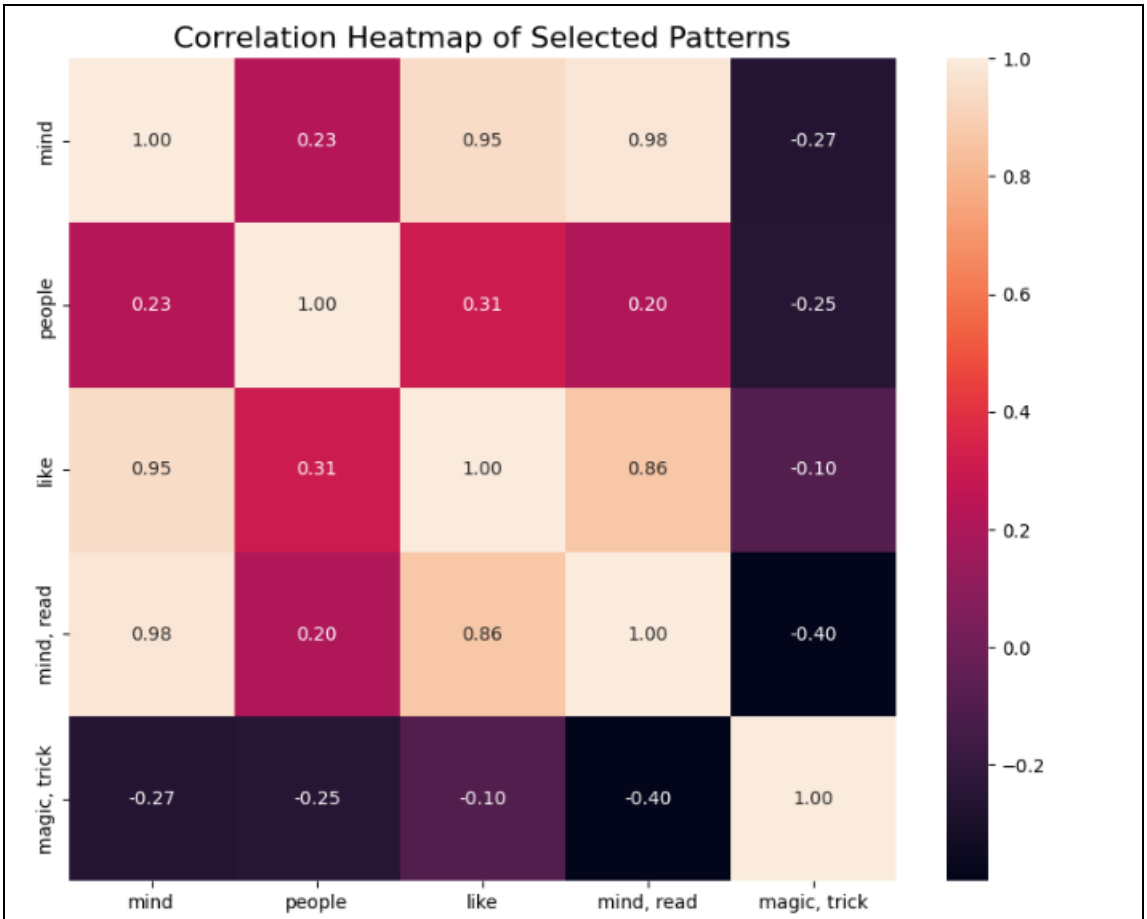


Figure 12: Correlation heatmap showing relationships between selected unigrams and bigrams across five chunks.

- Darker red/pink indicates strong positive correlation.
- Darker blue/purple indicates strong negative correlation.
- Values near neutral colors indicate weak or no correlation.

From the heatmap:

- Patterns like ‘mind’ and ‘mind, read’ are highly positively correlated (~0.98), suggesting they often increase or decrease together across chunks.
- ‘magic, trick’ shows negative correlation with other main words, indicating it behaves independently of core discussion words.
- ‘people’ shows weak correlation with most other patterns, reflecting moderate, independent variations.

Frequency Trend Line Plot

To complement the correlation analysis, a line plot was generated to visualize frequency changes over chunks:

```
plt.figure(figsize=(12,6))
for col in freq_df.columns:
    plt.plot(freq_df.index, freq_df[col], marker='o', linewidth=2, label=col)

plt.xlabel("Chunk", fontsize=14)
plt.ylabel("Frequency", fontsize=14)
plt.title("Pattern Frequency Over Time", fontsize=16)
plt.grid(True, alpha=0.3)
plt.legend(title='Patterns', fontsize=12)
plt.xticks(ticks=range(len(freq_df.index)), labels=[f"Chunk {i+1}" for i in range(len(freq_df.index))], fontsize=12)
plt.yticks(fontsize=12)
plt.show()
```

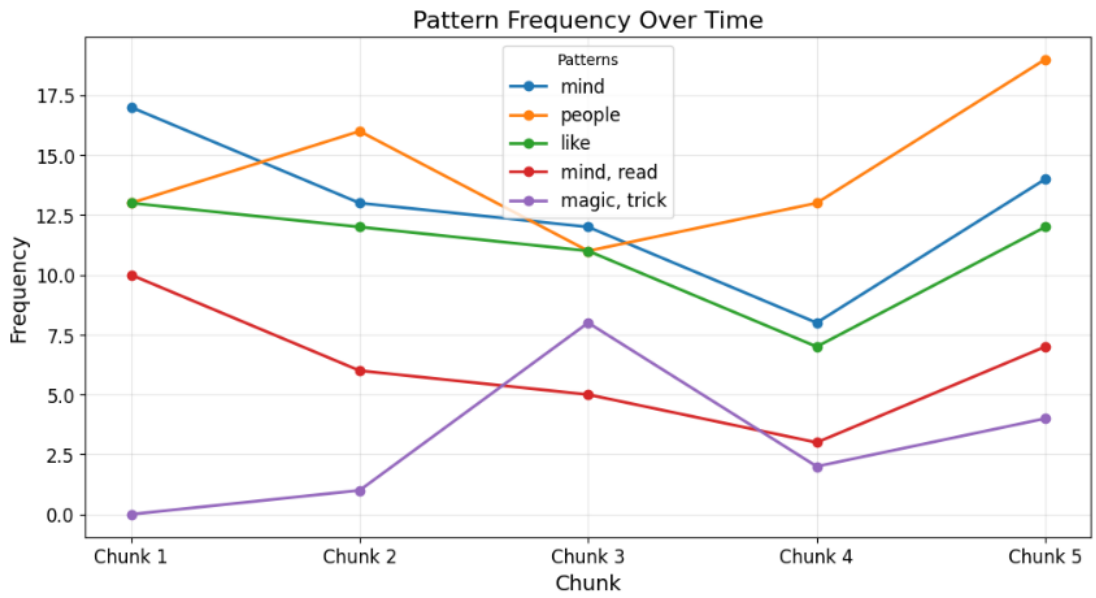


Figure 13: Line plot showing frequency trends of selected unigrams ('mind', 'people', 'like') and bigrams ('mind, read', 'magic, trick') across five chunks

- This plot allows easy identification of synchronized rises or falls among patterns.
- Diverging trends or sudden spikes/drops are easily visible, complementing numerical correlations.
- This line plot visualizes the frequency of different patterns over time across five chunks. It allows quick identification of synchronized trends, such as rises or falls in multiple patterns, as well as diverging behavior. For example, the frequency of "people" steadily

increases in the later chunks, while "magic, trick" shows independent fluctuations. Sudden spikes or drops, such as the peak of "magic, trick" in Chunk 3, are immediately noticeable, complementing numerical correlation analyses.

Interpretation

The combination of correlation matrices, heatmaps, and line plots provides a detailed understanding of pattern relationships:

1. Highly correlated patterns (e.g., 'mind' and 'mind, read') suggest tightly linked concepts or topics.
2. Negatively correlated patterns (e.g., 'magic, trick') behave independently, highlighting emergent themes.
3. Moderately correlated or weakly correlated patterns (e.g., 'people') indicate more diffuse influence across the discussion.
4. Temporal visualization clarifies how patterns rise, fall, or diverge, supporting deeper interpretation of discussion dynamics.

By combining quantitative correlation measures with trend visualization, this analysis effectively reveals both co-occurrence tendencies and temporal dynamics of key patterns across the dataset.

5.5 Reflection and Insight

Pattern trend Observations

1. **'mind'**

   The frequency of 'mind' shows noticeable fluctuations across different chunks. In later chunks, it increases, which may indicate that discussions are becoming more focused and thoughtful. This word also has a strong positive correlation with 'mind, read' (0.980), suggesting that as people mention 'mind,' they are often discussing reading or thinking deeply about the topic.

2. **'people'**

The word 'people' remains relatively stable, with only a slight upward trend. Its moderate correlation with 'like' (0.306) implies that mentions of 'people' may be related to general audience engagement or community-focused comments, rather than specific topic discussions.

3. **'like'**

‘Like’ shows high correlation with ‘mind’ (0.945) and ‘mind, read’ (0.865), indicating that liking behavior often aligns with topic-related content. Its frequency may vary over time depending on viewers’ engagement, such as liking comments when discussions become more thought-provoking. A low correlation with ‘magic, trick’ (-0.101) suggests that ‘like’ is less related to performance or entertainment topics.

4. **(‘mind,’ ‘read’)**

This pair has a strong positive correlation with ‘mind’ (0.980), confirming that the discussion is often theme-focused. Frequent co-occurrence of these words shows that users are concentrating on intellectual or content-related aspects rather than casual engagement. The negative correlation with ‘magic, trick’ (-0.397) further supports that this topic is distinct from performance-oriented discussions.

5. **(‘magic,’ ‘trick’)**

These terms tend to rise together, reflecting discussions about performance or entertainment. Their negative correlation with ‘mind’ (-0.266) and ‘mind, read’ (-0.397) suggests that when viewers focus on magical tricks, they are less engaged in intellectual or reading-related topics. Spikes in these words likely correspond to specific moments in videos or posts that highlight visual performance.

Correlation Insights

- **Mind and (mind, read):** Strong positive correlation confirms that intellectual or thematic discussions are consistent and focused.
- **Magic and (magic, trick):** Rise together, showing viewers’ attention to performance-based content.
- **Like:** Behaves differently from topic-related words, implying it reflects engagement behavior rather than the content theme.

6. Code Implementation

Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from itertools import combinations
import seaborn as sns
import ast
```

Explanation:

This cell sets up all the tools needed for reading, processing, analyzing and visualizing YouTube comments.

Load Data

```
df = pd.read_csv('/kaggle/input/datasets/priyaa652/cleaned-comments/cleaned_comments.csv')
df.head()
```

Explanation:

Load the cleaned comments CSV and check that it contains the columns comment_text and cleaned_tokens.

Ensure Tokens Are Lists

```
if isinstance(df['cleaned_tokens'].iloc[0], str):
    df['cleaned_tokens'] = df['cleaned_tokens'].apply(ast.literal_eval)
df['cleaned_tokens'].iloc[0]
```

Explanation:

Sometimes the tokens column is stored as a string. ast.literal_eval converts it into a proper Python list so that we can process tokens correctly.

Split Data into 5 Chunks

```
chunks = np.array_split(df, 5)
len(chunks)
```

Explanation:

This simulates comments coming in over time. Each chunk is analyzed separately to see how patterns evolve.

Initialize containers for Results

```
all_chunk_unigrams = []
all_chunk_bigrams = []
```

Explanation:

It will store results in these lists so that it can compare patterns across chunks later.

Process Each Chunk (Example: Chunk 1)

```
# Chunk 1
chunk = chunks[0]

# Extract transactions
transactions = chunk['cleaned_tokens'].tolist()

# Count unigrams
from collections import Counter
unigram_counts = Counter([token for row in transactions for token in row])
top_unigrams = unigram_counts.most_common(10)
all_chunk_unigrams.append(unigram_counts)

print("Top 10 Unigrams in Chunk 1:")
print(top_unigrams)

# Plot unigrams
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 8))
plt.bar([w for w, _ in top_unigrams], [c for _, c in top_unigrams])
plt.title('Top 10 Unigrams - Chunk 1', fontsize=16)
plt.xticks(rotation=45, fontsize=12)
plt.yticks(fontsize=12)
plt.show()

# Count co-occurring pairs
from itertools import combinations
bigram_counts = Counter()
for row in transactions:
    bigram_counts.update(combinations(sorted(set(row)), 2))
top_bigrams = bigram_counts.most_common(10)
all_chunk_bigrams.append(bigram_counts)

print("Top 10 Co-occurring Pairs in Chunk 1:")
print(top_bigrams)

# Plot co-occurring pairs
plt.figure(figsize=(10, 8))
plt.bar([str(pair) for pair, _ in top_bigrams], [c for _, c in top_bigrams])
plt.title('Top 10 Word Pairs - Chunk 1', fontsize=16)
plt.xticks(rotation=90, fontsize=12)
plt.yticks(fontsize=12)
plt.show()
```

Explanation:

- Count the frequency of single words (unigrams).
- Count co-occurring pairs of words (unordered bigrams) to see thematic links.
- Plot both for visual analysis.

Compare Patterns Across Chunks

```
TOP_N = 10

import pandas as pd

unigram_data = all_chunk_unigrams[:5]

all_top_unigrams = set()
for counts in unigram_data:
    top_words = [word for word, _ in counts.most_common(TOP_N)]
    all_top_unigrams.update(top_words)

unigram_summary = pd.DataFrame(index=sorted(all_top_unigrams))

for i, counts in enumerate(unigram_data):
    unigram_summary[f'Chunk {i+1}'] = [
        counts.get(word, 0) for word in unigram_summary.index
    ]

print("Unigram Comparison Across 5 Chunks:")
display(unigram_summary)
```

```
TOP_N = 10

import pandas as pd

bigram_data = all_chunk_bigrams[:5]

all_top_pairs = set()
for counts in bigram_data:
    top_pairs = [pair for pair, _ in counts.most_common(TOP_N)]
    all_top_pairs.update(top_pairs)

pair_summary = pd.DataFrame(index=sorted(all_top_pairs))

for i, counts in enumerate(bigram_data):
    pair_summary[f'Chunk {i+1}'] = [
        counts.get(pair, 0) for pair in pair_summary.index
    ]

print("Co-occurring Pair Comparison Across 5 Chunks:")
display(pair_summary)
```

Explanation:

- This creates a summary table of top patterns across chunks.
- It helps identify new words appearing, patterns disappearing, or stable trends.

Select Interesting Patterns and Build Frequency Data

```
# Step 1: Select 3–5 interesting patterns
patterns = [
    'mind',
    'people',
    'like',
    ('mind', 'read'),
    ('magic', 'trick')
]

# Step 2: Build frequency data

freq_data = {}

# Unigrams
for word in patterns:
    if isinstance(word, str):  # unigram
        freq_data[word] = [counts.get(word, 0) for counts in unigram_data[:5]]
    else:  # bigram tuple
        freq_data[f"{word[0]}, {word[1]}"] = [counts.get(word, 0) for counts in bigram_data[:5]]

# Step 3: Create DataFrame
freq_df = pd.DataFrame(freq_data, index=[f"Chunk {i+1}" for i in range(5)])

# Step 4: Display
print("Frequency Series of Selected Patterns Across 5 Chunks:")
display(freq_df)
```

Explanation:

- Builds time series data for selected words/pairs across chunks.
- This is used for correlation analysis.

Correlation Analysis and Visualization

```
# Compute Correlation Matrix
corr = freq_df.corr()

print("Correlation Matrix:")
print(corr)

#  Correlation Heatmap
plt.figure(figsize=(10,8))
sns.heatmap(freq_df.corr(), annot=True, fmt=".2f")
plt.title("Correlation Heatmap of Selected Patterns", fontsize=16)
plt.show()

# Frequency Line Plot
plt.figure(figsize=(12,6))
for col in freq_df.columns:
    plt.plot(freq_df.index, freq_df[col], marker='o', linewidth=2, label=col)

plt.xlabel("Chunk", fontsize=14)
plt.ylabel("Frequency", fontsize=14)
plt.title("Pattern Frequency Over Time", fontsize=16)
plt.grid(True, alpha=0.3)
plt.legend(title='Patterns', fontsize=12)
plt.xticks(ticks=range(len(freq_df.index)), labels=[f"Chunk {i+1}" for i in range(len(freq_df.index))], fontsize=12)
plt.yticks(fontsize=12)
plt.show()
```

Explanation:

- Correlation matrix identifies patterns that rise and fall together.
- Heatmap visually shows correlations.
- Line plot shows how each pattern's frequency evolves across chunks.

7 Results and Discussion

7.1 Observed Trends in Unigrams

- Stable Patterns:

Words like 'mind' and 'people' consistently appear across all chunks.

  - Insight: These terms indicate a central theme around thoughts, perceptions and social aspects in discussions. Their persistence suggests the audience's interest in core topics remained steady over time.

- Rising Trends:

Words like 'audience', 'name'and 'trick' show noticeable increases in the last chunks.

  - Insight: This suggests newer discussion topics, such as audience reactions, specific individuals' names, or tricks gained attention over time.

- Fluctuating Terms:

Words like ‘like’ and ‘magic’ fluctuate between chunks.

  - Insight: Engagement-focused words like ‘like’ respond to temporary attention spikes, while content specific words like ‘magic’ reflect episodic interest in certain content.

7.2 Co-occurring Pair Insights

- Emerging Pairs:
  - Pairs like (magic, trick), (mind, read), and (noah, trevor) increased in later chunks.
  - Interpretation: This show how content themes develop over time, with the audience gradually discussing specific concepts or people in more detail.
- Declining Pairs:
  - Early frequent pairs such as (actually, mind) and (actually, read) largely disappear in later chunks.
  - Interpretation: Early commentary focused on immediate reactions (‘actually’), but as discussion progressed, more thematic or narrative pairs gained prominence.
- Stable Pairs:
  - Pairs like (barack, obama) and (talk, ted) remain frequent across all chunks.
  - Interpretation: Core subjects and recurring content references maintain consistent attention, acting as anchor points in the discussion.

7.3 Frequency Patterns Across Chunks

- Mind related Terms:
  - ‘mind’, ‘mind, read’, and ‘magic, mind’ show moderate correlation, indicating discussion on mental or thought-related topics often intersects with reading or magical content.
- People focused Terms:
  - ‘people’, ‘like, people’, and ‘dont, people’ often correlate, highlighting discussions centered around human behavior, opinions or collective reactions.
- Emerging Content:
  - Terms like ‘magic, trick’ start low and rise in later chunks, indicating that specific content topics gradually attracted attention.
- Observation:

Frequency trends show that some topics fade while others emerge, reflecting the natural evolution of audience discussions over time.

7.4 Correlation Insights

Using the correlation matrix:

Pattern	Insight
mind & like (0.945)	Strong positive correlation: discussions about "mind" are often linked with approval or engagement (like).
mind & mind, read (0.980)	Very high correlation: reading-related discussions are strongly tied to mental or thought-focused topics.
magic, trick vs mind (-0.265)	Negative correlation: discussions about magic tricks tend to diverge from purely mental or reflective topics.
people & like (0.306)	Moderate positive correlation: social commentary correlates with engagement.
magic, trick vs others (-0.101 to -0.397)	Slight negative correlation with other terms: indicates magic/trick discussions form a relatively independent thematic cluster.

Interpretation:

- Positive correlations indicate topics that grow together over time, revealing intertwined themes.
- Negative correlations suggest thematic divergence: new content or niche topics that attract attention independently from the main discussion.
- Mental and social topics dominate the core conversation, while entertainment or niche topics emerge separately.

7.5 Analytical Summary of Topic Evolution

1. Core Themes:
    i. Mental, social, and engagement-related topics dominate across all chunks.
    ii. Examples: 'mind,' 'people,' 'like'.
2. Emerging Topics:
    i. Specific people, tricks, or audience-related terms appear later:'audience,' 'name', 'trick,' 'noah, trevor'.
    ii. These reflect evolving discussions as new content or specific references attract attention.
3. Shifting Co-occurrences:
    i. Early generic reactions decline (e.g., 'actually, mind'), while context-specific pairs grow ('magic, trick', 'name, people').

4. Relationship Insights:
    i. Correlations show which topics are often discussed together (mental & reading topics), and which evolve independently.

7.6 Challenges Faced During Analysis

1. Chunk-based approximation of time:
    i. Without exact timestamps, dividing comments into chunks may miss fine-grained temporal patterns.
2. Sparse co-occurring pairs:
    i. Many pairs appear infrequently, limiting statistical insights.
3. Interpreting correlations:
    i. High correlation does not imply causation; careful interpretation was required to avoid overstatement.

8. Conclusion

Analysis of word frequencies, co-occurring pairs, and correlations across five chunks shows clear trends in audience discussions. Core terms like ‘mind,’ ‘people,’ and ‘like’ remained dominant, reflecting sustained focus on mental and social themes. Over time, specific topics, such as ‘trick,’ ‘audience,’ and key names, gained prominence, indicating evolving engagement. Correlation patterns revealed strong links among reading- and mind-related terms, while niche topics like magic formed distinct clusters. Co-occurrence trends confirmed that some early general pairs faded, replaced by more content-specific combinations. Overall, the study highlights both the stability of key themes and the dynamic emergence of new topics, illustrating how audience interest shifts and interrelated ideas develop over time.